

Bases de Python

Documento de contexto para el alumnado

Material de apoyo para acompañar cada diapositiva de la presentación

Cómo usar este documento

Lee primero cada diapositiva y, después, su texto correspondiente. Lo ideal es ir observando la diapositiva, comprenderla y reflexionar sobre ella, para luego comparar, ampliar e interiorizar las ideas con ayuda de este documento. La finalidad no es memorizar el contenido palabra por palabra, sino adquirir el contexto suficiente para entender qué se está presentando, por qué es importante y cómo funciona python de arriba a abajo. Python será nuestro lenguaje base para aprender programación y para avanzar después hacia análisis de datos, IA y Big Data. Primero entendemos el lenguaje; luego elegimos herramientas cómodas para trabajar con él. En añadido, se incluye en código TutorialPython.py con el que se ha trabajado las imágenes de la diapositiva correspondiente.

Índice de bloques

- Diapositivas 1-5: introducción a Python y motivación.
- Diapositivas 6-12: instalación inicial, VS Code y primer programa.
- Diapositivas 13-19: estilo, PEP 8, extensiones y código limpio.
- Diapositivas 20-35: tipos, strings, números, condicionales, bucles y funciones.
- Diapositivas 36-37: transición a Anaconda, JupyterLab y entornos de trabajo.
- Apéndice final: links, descarga, instalación e inicialización de Anaconda y JupyterLab.

Contexto por diapositiva

Diapositiva 1 - Bases de Python

En esta primera diapositiva abrimos el bloque de Python. La idea es empezar desde cero, pero no desde la nada: Python será el lenguaje que usaremos como herramienta para trabajar programación, análisis de datos, inteligencia artificial y automatización.

Quiero que lo veáis como una navaja suiza: sirve para muchas cosas, pero al principio lo importante no es saber usar todas sus piezas, sino entender cómo se abre sin cortarse. Primero veremos la base del lenguaje y después lo conectaremos con entornos más cómodos para practicar, como JupyterLab.

Diapositiva 2 - Qué es Python

Python es un lenguaje de programación creado en 1991 por Guido van Rossum. Su popularidad viene, sobre todo, de que tiene una sintaxis clara y bastante legible. Eso no significa que programar sea siempre fácil, pero sí que Python intenta que el código se parezca menos a un jeroglífico y más a instrucciones entendibles.

Además, es un lenguaje multiparadigma: permite escribir código estructurado, orientado a objetos y también con ideas de programación funcional. Esto lo hace muy flexible. Se utiliza en desarrollo web, ciencia de datos, inteligencia artificial, automatización, videojuegos, scripts de administración, investigación científica y muchísimos campos más.

Otro punto clave es que funciona en Windows, macOS y Linux, y cuenta con una comunidad enorme. Esa comunidad ha creado miles de librerías que amplían el lenguaje. En la práctica, muchas veces no tendremos que inventar la rueda: podremos usar herramientas ya creadas, probadas y mantenidas por otros desarrolladores. La rueda está bien, pero reinventarla cada lunes cansa.

Diapositiva 3 - Por qué Python

En esta diapositiva resumo por qué vamos a trabajar con Python. El primer motivo es su comunidad: al ser tan usado, es fácil encontrar documentación, tutoriales, ejemplos, foros y soluciones a errores comunes. Esto es especialmente útil cuando empezamos, porque los errores llegarán. No es una amenaza, es una tradición de la programación.

El segundo motivo es su importancia en IA, Big Data y Data Science. Librerías como NumPy, pandas, scikit-learn, TensorFlow, PyTorch, Matplotlib o PySpark han convertido Python en uno de los lenguajes principales del ecosistema de datos.

También permite resolver tareas con menos código que otros lenguajes más verbosos. La comparación con C++ del meme tiene parte de broma y parte de verdad: Python sacrifica algo de control de bajo nivel, pero gana mucha claridad para escribir, probar y enseñar conceptos. Por eso es ideal para aprender y para prototipar soluciones.

Diapositiva 4 - Qué versión usamos

Aquí dejamos clara una decisión importante: trabajaremos con Python 3. Python 2 ya se considera legacy, es decir, una versión antigua que se mantiene solo en contextos muy concretos. Para aprender y para trabajar profesionalmente hoy, la referencia es Python 3.

Cuando busquemos Python en internet, conviene ir siempre a la fuente oficial: python.org. Descargar herramientas desde páginas raras puede acabar convirtiendo una clase de programación en una clase improvisada de ciberseguridad, y no queremos empezar así.

La versión exacta puede cambiar con el tiempo, pero la idea principal es mantenernos en una versión moderna y compatible con las librerías que vayamos a utilizar.

Diapositiva 5 - Primer contacto con la programación

Esta diapositiva usa un meme para reflejar algo muy real: al principio, programar parece una mezcla de símbolos, palabras nuevas y mensajes de error que parecen escritos por un oráculo enfadado. Después de unas semanas, empezamos a entender conceptos como variables, bucles, funciones, estructuras de datos y errores.

La idea importante es que la programación no se aprende de golpe. Se aprende por capas: primero se entiende qué es una variable, luego cómo usar condiciones, después bucles, funciones, estructuras de datos, y así sucesivamente. Cada concepto se apoya en el anterior.

Si al principio algo parece confuso, no significa que no sirváis para programar. Significa que estáis en la fase normal: la fase de mirar el código y pensar “esto lo he escrito yo, ¿no?”.

Diapositiva 6 - Paso 1: descarga desde Python.org

El primer paso consiste en entrar en la página oficial de Python y descargar el instalador correspondiente al sistema operativo. En la captura se ve la zona de descargas de Python, donde normalmente la propia web detecta si estamos en Windows, macOS o Linux.

Este paso es importante porque el intérprete de Python es el programa que realmente ejecuta nuestro código. Sin intérprete, escribiríamos código muy bonito, pero el ordenador lo miraría con la misma emoción que una tostadora mirando una novela.

En clase podemos empezar instalando Python directamente para entender el proceso básico, aunque más adelante usaremos Anaconda porque nos da un entorno más preparado para ciencia de datos.

Diapositiva 7 - Paso 2: instalación y PATH

En esta pantalla aparece el instalador de Python para Windows. Hay un detalle fundamental: marcar la opción Add python.exe to PATH. Esta opción permite que el sistema reconozca el comando python desde la terminal.

El PATH es una variable del sistema que indica dónde debe buscar programas ejecutables. Si Python no está en el PATH, podremos tenerlo instalado, pero al escribir python en la consola el sistema puede responder que no lo encuentra. Es el equivalente informático a tener las llaves en casa y seguir diciendo que se han perdido.

Después de marcar esa opción, podemos usar Install Now para hacer una instalación estándar.

Diapositiva 8 - Paso 3: comprobar la instalación

Después de instalar Python, comprobamos que todo funciona desde la terminal. El comando habitual es `python --version`. Si la instalación se ha hecho correctamente, veremos en pantalla la versión instalada.

Esta comprobación es una buena costumbre profesional. No damos por hecho que algo funciona solo porque hemos pulsado 'siguiente, siguiente, finalizar'. Primero verificamos. En programación, verificar ahorra mucho tiempo, y también evita mirar la pantalla con cara de tragedia griega cuando algo no arranca.

Si el comando no funciona, normalmente el problema está en el PATH, en que la instalación no se completó bien o en que el sistema usa otro comando, como `py` en Windows o `python3` en algunos sistemas.

Diapositiva 9 - Paso 4: editor de código o IDE

Una vez instalado Python, necesitamos un lugar cómodo donde escribir código. Podemos usar un editor de texto simple, pero para trabajar de verdad nos conviene un editor especializado o un IDE.

Un editor de código nos ayuda con coloreado de sintaxis, autocompletado, detección de errores, integración con terminal y organización de archivos. Un IDE, además, suele integrar herramientas más completas para depurar, ejecutar y gestionar proyectos.

En esta fase no buscamos complicarnos con mil herramientas. Buscamos un entorno que nos permita escribir, ejecutar y corregir código con comodidad.

Diapositiva 10 - Paso 5: Visual Studio Code

La diapositiva presenta VS Code, cuyo nombre completo es Visual Studio Code. Es un editor de código muy usado porque es gratuito, ligero, extensible y compatible con muchos lenguajes.

Para Python resulta especialmente útil porque permite instalar extensiones, abrir carpetas de proyecto, ejecutar scripts, trabajar con terminal integrada y visualizar errores. Aunque no es un IDE puro de Python desde el primer minuto, podemos convertirlo en un entorno muy completo mediante extensiones.

Aquí conviene distinguirlo de Visual Studio, que es otro producto distinto de Microsoft. VS Code es el editor que usaremos para escribir código de forma cómoda.

Diapositiva 11 - Paso 6: convertir VS Code en entorno para Python

En esta pantalla vemos que VS Code puede adaptarse a Python instalando una extensión. La extensión principal de Python añade soporte para ejecutar archivos, seleccionar intérpretes, depurar, detectar errores y trabajar con notebooks.

La idea es que VS Code por sí solo es como una mesa de trabajo vacía. Las extensiones son las herramientas que colocamos encima: destornillador, regla, lápiz, multímetro... o en nuestro caso, intérprete, autocompletado, formateador y depurador.

Una vez instalada la extensión, VS Code empieza a entender mejor nuestros archivos .py y nos ofrece ayudas específicas para programar en Python.

Diapositiva 12 - Paso 7: ejecutar el primer programa

Aquí ya aparece el primer contacto real con ejecución de código. Lo importante no es solo escribir una instrucción como `print('Hello world')`, sino entender el circuito completo: escribo un archivo, lo ejecuto con Python y veo una salida en la terminal.

La terminal puede parecer intimidante al principio, pero es una herramienta básica. Nos permite ejecutar programas, ver errores, instalar paquetes y comprobar resultados. En ciencia de datos y programación profesional, la terminal aparece constantemente.

El clásico Hello world es una tradición porque confirma que todo el entorno está funcionando: editor, archivo, intérprete y salida. Es pequeño, pero simbólicamente es el 'hola' del programador al ordenador. El ordenador normalmente responde sin emoción, pero cuenta.

Diapositiva 13 - Paso 8: PEPs y PEP 8

En esta diapositiva aparece el concepto de PEP. Las PEPs son documentos de propuesta y documentación relacionados con Python. Algunas explican nuevas características del lenguaje y otras establecen recomendaciones de estilo.

La más conocida para empezar es PEP 8, que recoge la guía de estilo para escribir código Python legible. Esto no es decoración: el código se lee muchas más veces de las que se escribe. Un código claro ayuda a detectar errores, colaborar con otros y volver a entender lo que hicimos semanas después.

PEP 8 nos habla, por ejemplo, de nombres de variables, espacios, indentación, longitud de línea y convenciones generales. Programar no es solo conseguir que funcione; también es conseguir que alguien pueda leerlo sin llorar por dentro.

Diapositiva 14 - Paso 8.1: aplicar PEP 8

La diapositiva muestra cómo aplicar formato al código desde VS Code. El formateo automático ayuda a que el código siga unas reglas de estilo sin tener que corregir cada espacio manualmente.

Esto no sustituye a entender PEP 8, pero sí nos ayuda a mantener consistencia. En programación profesional, la consistencia es muy importante: si cada persona del equipo escribe con un estilo distinto, el proyecto se vuelve más difícil de mantener.

Cuando VS Code indica que no hay formateador instalado, no es un drama; simplemente nos está diciendo que necesita una herramienta adicional para ordenar el código. El editor ayuda, pero no hace magia negra. Al menos no oficialmente.

Diapositiva 15 - Paso 8.2: extensión de formato

Aquí se ve la instalación de una extensión de formateo como autopep8. Esta herramienta toma nuestro código Python y lo reorganiza para acercarlo a las recomendaciones de PEP 8.

El formateador puede ajustar espacios, saltos de línea y otros detalles de estilo. Esto permite centrarnos más en la lógica del programa y menos en pequeños detalles visuales.

Aun así, conviene recordar que un formateador no arregla la lógica. Puede dejar el código muy bonito y perfectamente equivocado. La elegancia visual no garantiza que el programa haga lo correcto.

Diapositiva 16 - Paso 8.3: automatizar el formato

En esta pantalla se muestra la configuración para automatizar el formateo. La idea es que VS Code aplique el formato sin que tengamos que pedirlo manualmente cada vez.

Una práctica habitual es activar Format on Save, es decir, formatear al guardar. Así, cada vez que guardamos el archivo, el editor corrige el estilo del código automáticamente.

Esta costumbre es muy útil porque evita acumular desorden. En programación, el desorden pequeño se convierte rápido en selva. Y una selva de código sin machete no es un sitio agradable.

Diapositiva 17 - Paso 9: atajos y comandos de VS Code

La diapositiva menciona el atajo Shift + Control + P. Este atajo abre la paleta de comandos de VS Code, una especie de buscador interno desde el que podemos ejecutar muchas acciones: formatear documento, seleccionar intérprete, abrir configuración, instalar extensiones o cambiar preferencias.

Aprender algunos atajos acelera mucho el trabajo. No hace falta memorizar todos al principio, pero sí conviene conocer los más importantes. La paleta de comandos es especialmente útil porque nos permite buscar acciones sin navegar por todos los menús.

Pensad en ella como el buscador de Google, pero dentro del editor. Si no sabéis dónde está algo, probablemente la paleta lo encuentre antes que vosotros. Y sin juzgaros.

Diapositiva 18 - Paso 10: implementaciones de Python

En esta diapositiva aparecen CPython, Jython, IronPython y PyPy. Esto sirve para distinguir entre el lenguaje Python y sus implementaciones. Python es el lenguaje; la implementación es el programa concreto que ejecuta ese lenguaje.

CPython es la implementación más común y la que normalmente usamos cuando instalamos Python desde python.org. Está escrita en C y es la referencia principal del ecosistema.

Jython permite integrar Python con Java; IronPython se relaciona con el entorno .NET; y PyPy destaca por intentar mejorar el rendimiento mediante técnicas como compilación Just-In-Time. Para empezar, nos basta con CPython, pero es bueno saber que Python no es una sola pieza monolítica.

Diapositiva 19 - Paso 11: nombres de variables y código limpio

Aquí empezamos con una de las bases del código limpio: nombrar bien las variables. Una variable debe tener un nombre descriptivo, en minúsculas y fácil de entender. En Python se suele usar snake_case, es decir, palabras separadas por guiones bajos: nombre_usuario, precio_total o temperatura_media.

No se usan espacios dentro de nombres de variables. También conviene dejar espacios alrededor del operador de asignación: `edad = 22` se lee mejor que `edad=22`. Parece un detalle pequeño, pero estos detalles hacen que el código sea más mantenible.

Una variable llamada `x` puede servir en un ejemplo matemático corto; en un proyecto real, nombres como `total_ventas` o `numero_clientes` explican mucho más. El objetivo es que el código cuente parte de la historia por sí mismo.

```
precio_total = 125.50
numero_clientes = 42
temperatura_media = 23.7
```

Diapositiva 20 - Paso 12: tipos básicos de variables

En esta diapositiva entramos en los tipos básicos: `bool`, `int`, `float` y `string`. Cada tipo representa una clase de dato diferente.

Un `bool` representa valores lógicos: `True` o `False`. Un `int` representa números enteros, como 3, 10 o -25. Un `float` representa números decimales, como 3.14. Un `string`, o `str`, representa texto, como 'Hola' o 'Python'.

Entender los tipos es fundamental porque las operaciones dependen de ellos. No es lo mismo sumar números que concatenar texto. Python es flexible, pero no lee la mente. Si mezclamos tipos sin pensar, aparecerán errores, y los errores suelen ser muy sinceros, aunque no siempre muy amables.

```
activo = True          # bool
edad = 22              # int
precio = 19.99         # float
nombre = "Python"     # str
```

Diapositiva 21 - Paso 12.1: strings y comillas

Esta pantalla introduce las cadenas de texto y el uso de comillas. En Python podemos crear strings usando comillas simples o dobles. Por ejemplo, 'Hola' y "Hola" representan texto.

También aparecen las triples comillas, que permiten escribir textos en varias líneas. Esto es útil cuando queremos guardar párrafos completos, mensajes largos o bloques de texto sin tener que cerrar y abrir comillas en cada línea.

La elección entre comillas simples y dobles suele ser cuestión de estilo, salvo cuando el texto contiene una comilla dentro. En ese caso, elegir bien las comillas ayuda a evitar errores de sintaxis.

Diapositiva 22 - Paso 12.2: longitud e índices de un string

Aquí vemos que un string no es solo texto: también se comporta como una secuencia de caracteres. Por eso podemos medir su longitud con `len()` y acceder a posiciones concretas mediante índices.

En Python, los índices empiezan en 0. Esto significa que el primer carácter de una cadena está en la posición 0, no en la 1. Al principio parece una decisión diseñada para molestar, pero en informática tiene mucha lógica por cómo se representan las posiciones en memoria y estructuras de datos.

También podemos usar índices negativos. El índice -1 representa el último carácter, -2 el penúltimo, y así sucesivamente. Esto resulta muy cómodo para acceder al final de una cadena sin calcular su longitud manualmente.

Diapositiva 23 - Paso 12.3: slicing de strings

Esta diapositiva continúa con los strings y muestra el slicing, que permite extraer fragmentos de una cadena. La forma general es `texto[inicio:fin]`. El punto importante es que el inicio se incluye y el fin no se incluye.

Por ejemplo, `texto[0:3]` toma los caracteres desde la posición 0 hasta antes de la posición 3. También podemos omitir el inicio o el fin: `texto[:3]` toma desde el principio hasta antes de la posición 3, y `texto[3:]` toma desde la posición 3 hasta el final.

El slicing es una herramienta muy útil porque aparece también en listas, arrays de NumPy, series de pandas y muchas estructuras de datos. Aprenderlo bien aquí nos ahorrará dolores de cabeza después.

```
texto = "Python"
texto[0]    # primer carácter
texto[-1]   # último carácter
texto[0:3]  # fragmento desde 0 hasta antes de 3
```

Diapositiva 24 - Paso 12.4: diagonal invertida y caracteres especiales

La diagonal invertida, o backslash, se usa para representar caracteres especiales dentro de strings. Por ejemplo,

`\n` representa un salto de línea y `\t` representa una tabulación.

También puede usarse para escapar comillas dentro de una cadena. Escapar significa decirle a Python: 'este carácter forma parte del texto, no lo interpretes como cierre de la cadena'.

Este concepto es importante porque los strings no siempre contienen texto simple. Pueden contener rutas de archivos, saltos de línea, tabulaciones, comillas internas o caracteres especiales. La diagonal invertida es pequeña, pero cuando falta, el error aparece con entusiasmo.

Diapositiva 25 - Paso 12.5: métodos de strings

En esta diapositiva aparecen métodos de strings como `upper()`, `lower()`, `title()`, `strip()`, `find()` y `replace()`. Un método es una función asociada a un objeto. En este caso, son acciones que podemos aplicar sobre un texto.

Por ejemplo, `upper()` convierte el texto a mayúsculas, `lower()` a minúsculas, `title()` capitaliza palabras, `strip()` elimina espacios al principio y al final, `find()` busca una subcadena y `replace()` sustituye una parte del texto por otra.

También aparece el uso de f-strings, una forma cómoda de construir mensajes mezclando texto y variables. Las f-strings son muy útiles para imprimir resultados legibles sin concatenaciones eternas. Nuestro futuro yo agradecerá mensajes claros.

Diapositiva 26 - Paso 12.5: variables numéricas y operaciones

Esta pantalla trabaja con variables numéricas y operaciones básicas. Vemos tipos como `int`, `float` y `complex`, y también funciones como `type()` o `isinstance()`, que sirven para comprobar el tipo de un dato.

A nivel de operaciones, Python permite sumar, restar, multiplicar, dividir, hacer división entera, calcular potencias, obtener el módulo, redondear y usar funciones como `abs()`, `pow()` o `divmod()`. Estas operaciones son la base para cualquier cálculo posterior en análisis de datos.

Aquí conviene entender muy bien la diferencia entre `/` y `//`. La primera realiza división normal; la segunda realiza división entera. En programación, un símbolo pequeño puede cambiar bastante el resultado. Python no hace drama, simplemente calcula otra cosa.

Diapositiva 27 - Paso 12.6: condicionales

Los condicionales permiten que un programa tome decisiones. Con `if`, `elif` y `else` podemos ejecutar bloques distintos según se cumpla o no una condición.

En la diapositiva se ve un ejemplo con temperatura: si supera cierto valor, el programa imprime que hace calor; si está en otro rango, indica buen tiempo; y si no, indica frío. También aparece un ejemplo con edad y operadores lógicos como `and` y `or`.

Esta estructura es fundamental porque convierte el programa en algo dinámico. Ya no ejecuta siempre exactamente lo mismo: responde a los datos. En IA y análisis de datos, muchas decisiones automáticas parten de esta idea básica, aunque luego se vuelvan mucho más complejas.

```
temperatura = 25

if temperatura > 30:
    print("Hace calor")
elif temperatura > 20:
    print("Hace buen tiempo")
else:
    print("Hace frío")
```

Diapositiva 28 - Paso 12.7.1: bucles simples

Los bucles for permiten repetir instrucciones un número determinado de veces o recorrer una colección de elementos. En la diapositiva aparece range(), que genera secuencias de números.

Por ejemplo, range(6) produce valores desde 0 hasta 5. También podemos indicar inicio, fin y salto: range(1, 10, 2) permite recorrer números empezando en 1, hasta antes de 10, saltando de dos en dos.

Los bucles son una de las estructuras más importantes de la programación. Nos permiten automatizar tareas repetitivas. Si algo hay que hacerlo cien veces, lo ideal no es copiar y pegar cien líneas: lo ideal es usar un bucle y conservar nuestra dignidad.

Diapositiva 29 - Paso 12.7.2: bucles anidados

Un bucle anidado es un bucle dentro de otro bucle. En la diapositiva se ve un ejemplo con dos variables, x e y, donde por cada valor de x se recorren varios valores de y.

Esto es útil para recorrer combinaciones, tablas, matrices, coordenadas o estructuras bidimensionales. Por ejemplo, una tabla tiene filas y columnas; una imagen tiene alto y ancho; una matriz tiene índices en dos dimensiones.

Los bucles anidados son potentes, pero hay que usarlos con cuidado. Si un bucle repite 1.000 veces y dentro hay otro que repite 1.000 veces, el total son 1.000.000 de iteraciones. Aquí empieza la diferencia entre 'funciona' y 'me da tiempo a merendar mientras funciona'.

Diapositiva 30 - Paso 12.7.3: bucles while

El bucle while repite un bloque mientras una condición sea verdadera. A diferencia del for, no necesita saber de antemano cuántas veces va a repetirse.

En la diapositiva aparecen dos usos típicos. Uno reduce un número mientras sea mayor que cero. Otro pide al usuario que escriba una palabra hasta que indique salir. Este tipo de estructura es muy común en menús, validaciones y programas interactivos.

La clave es asegurarnos de que la condición pueda dejar de cumplirse en algún momento. Si no, el programa seguirá ejecutándose sin parar, y entonces ya no tenemos un bucle: tenemos una criatura descontrolada.

Diapositiva 31 - Paso 12.7.3: bucles infinitos y break

Esta diapositiva muestra una forma habitual de controlar un bucle potencialmente infinito: while True junto con break. while True crea un bucle que, en principio, no termina nunca. La salida se controla desde dentro con una condición.

En el ejemplo, el programa pide una contraseña o palabra de salida y, si coincide con el valor esperado, ejecuta break para salir del bucle. Esto es útil cuando no sabemos cuántas veces tendrá que intentar algo el usuario.

La instrucción `break` debe usarse con claridad. Es una salida de emergencia del bucle. Como toda salida de emergencia, conviene saber dónde está antes de que empiece el incendio.

Diapositiva 32 - Paso 12.8.1: funciones

Las funciones permiten agrupar instrucciones bajo un nombre. En Python se definen con `def`, seguido del nombre de la función, paréntesis y dos puntos. El bloque indentado contiene lo que la función hace.

En el ejemplo, una función `greet()` imprime varios mensajes. Después, llamamos a la función escribiendo su nombre con paréntesis. Definir una función no la ejecuta automáticamente; solo la deja preparada para usarla.

Las funciones son esenciales porque evitan repetir código, organizan mejor los programas y permiten dividir problemas grandes en piezas más pequeñas. Programar bien consiste muchas veces en trocear un problema hasta que deja de dar miedo.

Diapositiva 33 - Paso 12.8.2: funciones con argumentos

Aquí vemos funciones que reciben argumentos, también llamados parámetros. Un parámetro permite que la función trabaje con datos diferentes cada vez que la llamamos.

Por ejemplo, una función que saluda puede recibir un nombre y construir un mensaje personalizado. También aparecen parámetros opcionales, como un apellido que puede pasarse o no. Si no se proporciona, la función usa un valor por defecto.

Esto hace que las funciones sean flexibles. En lugar de crear una función distinta para cada nombre, creamos una sola función que recibe información variable. Es una de las ideas más importantes para escribir código reutilizable.

Diapositiva 34 - Paso 12.8.3: funciones con return

La palabra `return` permite que una función devuelva un resultado. No es lo mismo imprimir algo con `print()` que devolverlo con `return`. `print()` muestra información en pantalla; `return` entrega un valor para que el programa pueda seguir usándolo.

En la diapositiva aparece una función `add(n1, n2)` que devuelve la suma de dos números. Luego guardamos ese resultado en una variable y lo imprimimos. Esta estructura es muy común: cálculo dentro de una función, devuelvo el resultado y lo uso fuera.

Entender `return` es fundamental para avanzar. Una función sin `return` puede servir para mostrar mensajes o ejecutar acciones; una función con `return` puede integrarse en cálculos, algoritmos y procesos más complejos.

```
def sumar(n1, n2):
    return n1 + n2

resultado = sumar(5, 3)
print(resultado)
```

Diapositiva 35 - Paso 12.8.3: argumentos con asterisco

Esta diapositiva introduce los argumentos con asterisco, normalmente escritos como `*args`. Sirven para crear funciones que aceptan un número variable de argumentos.

En el ejemplo, la función `multiply(*numbers)` recibe varios números, los recorre con un bucle y acumula el resultado multiplicándolos. Así podemos llamar a la función con dos, tres, cuatro o más valores sin cambiar su definición.

Este recurso es útil cuando no sabemos cuántos datos recibirá una función. Más adelante, al trabajar con librerías y funciones avanzadas, veremos que este patrón aparece bastante. De momento, lo importante es entender que el asterisco empaqueta varios argumentos en una colección.

```
def multiplicar(*numeros):
    total = 1
    for numero in numeros:
        total *= numero
    return total

resultado = multiplicar(2, 3, 4)
```

Diapositiva 36 - Cambio de entorno: de VS Code a Anaconda

Después de dar los primeros pasos con VS Code, la presentación propone pasar a Anaconda. El motivo es que Anaconda viene preparada para trabajar con ciencia de datos y enseñanza de forma más visual, especialmente mediante notebooks.

Un notebook permite combinar código, texto, resultados, tablas y gráficos en un mismo documento. Esto lo hace muy cómodo para aprender, explicar procesos, resolver ejercicios y documentar análisis. Para IA, Big Data y Data Science, este formato es especialmente práctico.

Quien quiera podrá seguir usando VS Code, y también existe Google Colab, que permite trabajar desde el navegador en la nube. Colab puede dar acceso a recursos como GPU o TPU en determinados casos. Aun así, para clase usaremos Anaconda y JupyterLab como entorno principal porque nos da una estructura clara y local.

Diapositiva 37 - Anaconda Navigator y JupyterLab

La última pantalla muestra Anaconda Navigator, una interfaz gráfica desde la que podemos lanzar herramientas como JupyterLab, Jupyter Notebook, Spyder o consolas interactivas.

JupyterLab será nuestro entorno principal para trabajar con notebooks. Desde Navigator podremos abrirlo pulsando Launch, sin necesidad de memorizar comandos al principio. Más adelante también podremos abrirlo desde terminal con `jupyter lab`, que es la forma habitual en muchos entornos profesionales.

La idea es que Anaconda nos simplifique la parte de instalación y gestión inicial. Así podremos centrarnos antes en programar, analizar datos y entender conceptos, en lugar de pelear con dependencias desde el minuto uno. Ya habrá tiempo para pelearse con dependencias; nadie escapa para siempre.

Apéndice - Descarga, instalación e inicio de Anaconda y JupyterLab

Para continuar el curso con notebooks, vamos a usar Anaconda y JupyterLab. Anaconda nos instala un entorno preparado para ciencia de datos y JupyterLab nos permite trabajar con código, texto y resultados en un mismo espacio.

Descarga oficial de Anaconda

Para instalar el entorno que vamos a utilizar, debéis ir a la página oficial de Anaconda y descargar Anaconda Distribution para vuestro sistema operativo. Es importante hacerlo desde la web oficial, no desde páginas de terceros.

En la página de descarga puede aparecer un formulario de registro o una opción para continuar con la descarga. Seguid las indicaciones de la web y elegid el instalador correspondiente a Windows, macOS o Linux.

Instalación básica

Ejecutad el instalador descargado y seguid los pasos por defecto. En Windows, normalmente recomiendo instalarlo solo para vuestro usuario, salvo que tengáis una razón concreta para instalarlo para todos los usuarios del equipo.

Si el instalador pregunta si queréis añadir Anaconda al PATH, lo más habitual es no marcar esa opción y usar Anaconda Navigator o Anaconda Prompt. Así evitamos conflictos con otras instalaciones de Python.

Abrir Anaconda Navigator

Después de instalar, buscad Anaconda Navigator en el menú de inicio o en las aplicaciones del sistema. Navigator es la interfaz gráfica desde la que podremos abrir JupyterLab y gestionar entornos de trabajo.

En la pantalla principal deberíais ver tarjetas de aplicaciones. Buscad JupyterLab y pulsad Launch. Si aparece como no instalado, Navigator debería permitir instalarlo desde la propia interfaz.

Inicializar JupyterLab

Al abrir JupyterLab, se iniciará un servidor local y normalmente se abrirá una pestaña en el navegador. Aunque lo veamos en el navegador, no significa que estemos necesariamente trabajando en internet: JupyterLab se ejecuta en nuestro propio ordenador y el navegador actúa como interfaz.

Dentro de JupyterLab, cread un notebook nuevo seleccionando un kernel de Python. A partir de ahí podréis escribir celdas de código, ejecutarlas y ver resultados debajo de cada celda. Esta forma de trabajo es ideal para aprender, probar pequeñas piezas de código y documentar lo que hacemos.

```
# Desde Anaconda Prompt o terminal:
jupyter lab
```

Alternativa desde terminal

También podéis abrir JupyterLab desde Anaconda Prompt o desde una terminal configurada. Primero conviene situarse en la carpeta donde queréis trabajar usando `cd`. Después se ejecuta el comando `jupyter lab`.

Es recomendable abrir JupyterLab desde una carpeta de trabajo propia, no desde la raíz del sistema como `C:\` o `/`. Así reducimos el riesgo de crear, mover o modificar archivos en lugares que no debemos.

```
cd ruta/de/mi/carpeta/de/trabajo
jupyter lab
```

Cerrar correctamente

Cuando terminéis de trabajar, guardad el notebook. Después podéis cerrar JupyterLab desde la interfaz y detener el servidor desde la terminal con Ctrl + C si lo habéis abierto por comandos. Si lo abristeis desde Navigator, podéis cerrar la aplicación desde ahí.

La costumbre profesional es guardar, cerrar y ordenar. Parece básico, pero muchos dramas informáticos empiezan con un archivo llamado prueba_final_final_ahora_si_bueno.ipynb.

Recordatorio: Aunque JupyterLab se abra en el navegador, el servidor se ejecuta localmente en vuestro ordenador cuando lo lanzáis desde Anaconda. No es lo mismo que Google Colab, que trabaja en la nube.

Enlaces oficiales y recursos útiles

- Python.org: <https://www.python.org/>
- PEP 8 - Guía de estilo para código Python: <https://peps.python.org/pep-0008/>
- Visual Studio Code: <https://code.visualstudio.com/>
- Descarga oficial de Anaconda Distribution: <https://www.anaconda.com/download>
- Documentación de Anaconda Navigator:
<https://www.anaconda.com/docs/tools/anaconda-navigator/main>
- Cómo abrir JupyterLab con Anaconda:
<https://www.anaconda.com/docs/getting-started/guides/how-to-open-jupyterlab>
- Documentación oficial: iniciar JupyterLab:
https://jupyterlab.readthedocs.io/en/stable/getting_started/starting.html
- Proyecto Jupyter: <https://jupyter.org/>
- Google Colab: <https://colab.research.google.com/>